Week 14

AI 디버깅 · 테스트 · 리팩토링

AI활용코딩연습(ZBE135)

이영호 교수 · 국립목포대학교 컴퓨터학부

지난주 돌아보기

```
fig, ax = plt.subplots()
ax.barh(share.index, share.values, color=colors)
ax.set_title("아메리카노가 매출의 약 4분의 1을 차지한다", loc="left")
audit_chart(ax)       # 그래프도 코드로 점검할 수 있다
```

오늘: 코드 전체를 **진단하고, 테스트하고, 다듬는** 방법

오늘의 목표

1. **재현 → 범위 좁히기 → 가설 → 확인 → 수정 → 회귀 테스트**
2. 오류가 나지 않는 **조용한 버그**를 작은 데이터로 찾기
3. 기대 결과를 먼저 적는 **테스트**로 수정 · 리팩토링을 안전하게
4. 이름 · 상수 · 함수 추출 · 중복 제거 · 부수효과 제거로 **리팩토링**
5. AI 디버깅: **좋은 질문**, **가짜 수정** 경계

수업 흐름

구간	시간	활동
도입 · 개념	20분	디버깅 절차, 트레이스백, 조용한 버그
안내 실습	45분	결함 있는 주간 보고서 진단 · 수정
독립 · 짝 실습	35분	리팩토링, AI 디버깅
공유 · 점검	20분	최종 프로젝트 점검표, 과제 안내

이번 주 Colab 실습 열기

디버깅 6단계

단계	할 일
① 재현	같은 입력 → **항상** 같은 문제 (작은 입력, 시드 고정)
② 범위 좁히기	중간값 `print`, 반으로 나누기
③ 가설	"이 값이 문자열이라서?"
④ 확인	`assert`, `type()`, `shape`로 실험
⑤ 수정	**원인**을 고친다 (증상만 가리지 않는다)
⑥ 회귀 테스트	같은 버그가 다시 생기지 않게 테스트 추가

트레이스백은 아래에서 위로

```
Traceback (most recent call last):
  File "...", line 10, in <module>
    average_price(["2,500", "3,200", "3천원"])
  File "...", line 8, in average_price
    prices = load_prices(rows)
  File "...", line 2, in load_prices
    return [parse_price(r) for r in rows]
  File "...", line 5, in parse_price
    return int(text.replace(",", ""))
ValueError: invalid literal for int() with base 10: '3천원'
```

마지막 줄: **무엇이** → 바로 위: **어디서** → 더 위: **누가 불렀나**

조용한 버그

오류 없이 **틀린 값만** 나온다 → 가장 위험

유형	예
경계	`range(1, n)` 이 n을 빠뜨림
분모 착오	영업일 대신 7로 나눔
집계 단위	수량 대신 건수
부수효과	함수가 입력에 열을 추가
초기화 위치	반복 안에서 `total = 0`
NaN 비교	`x == np.nan`

무기: 손으로 답을 아는 작은 데이터

날짜	주차	메뉴	가격	수량
03-03	10	A	1000	1
03-03	10	B	2000	1
03-04	10	A	1000	1
03-10	11	B	2000	3
03-10	11	A	1000	1
03-10	11	A	1000	1

기대: 10주 → (2,000원, A) · 11주 → (8,000원, **B**: 수량 3 vs 건수는 A가 많음)

AI의 주간 보고서

```
def weekly_report_ai(cafe):
    cafe["revenue"] = cafe["unit_price"] * cafe["qty"]
    cafe["week"] = cafe["date"].dt.isocalendar().week
    result = {}
    for w in range(cafe["week"].min(), cafe["week"].max()):
        sub = cafe[cafe["week"] == w]
        avg = sub["revenue"].sum() / 7
        top = sub["menu"].value_counts().index[0]
        result[w] = (round(avg), top)
    return result
```

작은 데이터 결과: `{10: (571, 'A')}` — 버그가 몇 개일까?

테스트 팩토리

```
def make_tests(fn):
    def test_values():
        assert fn(make_tiny()) == EXPECTED
    def test_last_week_included():
        assert 11 in fn(make_tiny())
    def test_input_unchanged():
        tiny = make_tiny(); before = tiny.copy()
        fn(tiny)
        assert tiny.equals(before)
    return [test_values, test_last_week_included, test_input_unchanged]

run_tests(make_tests(weekly_report_ai))      # 0/3
```

같은 테스트로 **AI 버전과 고친 버전을 모두** 검사

버그 4개

버그	원인	수정
마지막 주 누락	`range(min, max)` 끝은 미만	실제 주차 `unique()`
일평균이 작음	7로 나눔	`sub["date"].nunique()`
인기 메뉴가 다름	건수 기준	`groupby("menu")["qty"].sum().idxmax()`
입력이 바뀜	`cafe["revenue"] = ...`	`cafe.assign(...)`

고친 버전: **3/3 통과**, 전체 데이터 16주 (AI 버전 15주)

리팩토링: 결과는 그대로

기법	전 → 후
이름	`d`, `x2` → `daily`, `ice_ratio`
상수	`* 0.3`, `< 12` → `WEIGHTS`, `MIN_ATTENDANCE`
함수 추출	긴 함수 → 작은 함수들
중복 제거	같은 코드 3번 → 반복 · 함수
부수효과 제거	입력 수정 → 새 객체 반환
early return	깊은 중첩 → 예외 먼저

테스트가 먼저 있어야 안전하다

리팩토링 전

```
def f(df):
    r = {}
    x = df[df["category"] == "커피"]
    r["커피"] = (len(x), int((x["unit_price"] * x["qty"]).sum()),
                 round((x["unit_price"] * x["qty"]).sum() / len(x)))
    x = df[df["category"] == "논커피"]
    r["논커피"] = (len(x), int((x["unit_price"] * x["qty"]).sum()), ...)
    x = df[df["category"] == "디저트"]
    r["디저트"] = (...)
    return r
```

리팩토링 후

```
CATEGORIES = ["커피", "논커피", "디저트"]

def summarize_group(rows):
    """주문 행들의 (주문 건수, 매출 합계, 건당 평균 매출)."""
    revenue = rows["unit_price"] * rows["qty"]
    return len(rows), int(revenue.sum()), round(revenue.sum() / len(rows))

def category_summary(cafe, categories=CATEGORIES):
    return {cat: summarize_group(cafe[cafe["category"] == cat])
            for cat in categories}

assert category_summary(CAFE) == f(CAFE)     # 결과 동일 확인
```

실습 정리 (Level 1 → 3)

Level 1 트레이스백 읽기, 초기화 위치 버그 재현 · 중간값 출력

Level 2 `summarize_group`에 빈 데이터 → `(0, 0, 0)`으로 고치고 테스트 추가 · `groupby`로 다시 쓰고 결과 대조

Level 3 주간 보고서: 작은 데이터 → 테스트 팩토리 → 원인 4개 → 수정 → 회귀 테스트

AI에게 디버깅을 맡길 때

나쁜 질문	좋은 질문
"이상해요, 고쳐 주세요" + 300줄	**최소 재현 코드** 10~20줄
	입력 · 기대 결과 · 실제 결과
	오류라면 **트레이스백 전체**
	이미 확인한 것
	"원인을 먼저 설명, 수정은 최소한으로"

좋은 질문 예

```
아래 함수는 주별 일평균 매출을 구해야 합니다.
[최소 재현 코드 + make_tiny() 데이터]
기대 결과: {10: (2000, 'A'), 11: (8000, 'B')}
실제 결과: {10: (571, 'A')}
11주가 빠지고 10주 평균이 작습니다.
원인을 먼저 줄 단위로 설명하고, 수정은 최소한으로 해 주세요.
입력 DataFrame은 바꾸면 안 됩니다.
```

가짜 수정을 경계하라

```
def weekly_report_cheat(cafe):
    if len(cafe) == 6:                          # 테스트 데이터일 때만
        return {10: (2000, "A"), 11: (8000, "B")}
    return weekly_report_ai(cafe.copy())
```

- 테스트는 **모두 통과**, 실제 데이터는 **여전히 틀림**
- 대책: ① 고친 **코드를 읽는다** ② **두 번째 테스트 데이터** ③ AI가 **테스트를 고치자고** 하면 의심

함수	두 번째 데이터 결과
cheat	❌
fixed	✅

최종 프로젝트 점검표 (릴리스 후보)

- [ ] **다시 시작 및 모두 실행** → 오류 없음
- [ ] 핵심 계산 함수마다 **작은 데이터 테스트** 1개 이상
- [ ] 정제 · 집계 함수는 **입력을 바꾸지 않음**
- [ ] 매직 넘버는 상수, 함수 이름은 하는 일을 설명
- [ ] 같은 코드 3번 이상 반복 없음
- [ ] AI 코드는 모두 **설명할 수 있음**, AI 활용 기록 갱신

이번 주 과제 (10점)

문제	내용	배점
1	`score_stats` 버그 3개 + 버그 보고서	2
2	`my_tests`로 `clean_price` 버그 함수 3개 잡기	3
3	`payment_report` 리팩토링 (결과 동일, 도우미 함수, 10줄 이하)	3
4	`daily_ice_ratio` pandas 버그 (AI 협업)	2

자기 점검

1. 디버깅 6단계를 순서대로 말하면?
2. 트레이스백에서 가장 먼저 볼 줄은?
3. "조용한 버그"를 찾는 가장 좋은 방법은?
4. 리팩토링 전에 테스트가 있어야 하는 이유는?
5. AI의 수정이 가짜인지 확인하는 방법 세 가지는?

정리와 다음 주

오늘	핵심
디버깅	재현 → 좁히기 → 가설 → 확인 → 수정 → 회귀 테스트
테스트	손계산 가능한 작은 데이터, 테스트 팩토리
리팩토링	결과 동일 assert + 이름 · 상수 · 함수 · 중복
AI	최소 재현 질문, 가짜 수정 경계

다음 주 · Week 15 최종 프로젝트 발표와 재현성